Bygg en Terminalemulator med Telnet: Uppgiftsbeskrivning
Kungliga Tekniska Högskolan (KTH) 12 Mars 2020
Befattningsutbildning till Cybersoldat HT2020

Kim Hammar, Viktor Engström, Marcus Schmidt Birgersson, Annika Andreasson

Introduktion

Telnet är ett nätverksprotokoll från 70-talet som möjliggör simpel textbaserad kommunikation mellan datorer uppkopplade på ett gemensamt nätverk. Till exempel, antag att du befinner dig på HOST1 och vill logga in och få tillgång till terminalen på HOST2. I ett sådant scenario kan följande Telnet kommando användas, förutsatt att en telnetserver körs på HOST2 och att HOST1 har tillgång till en Telnetklient:

```
telnet hostname
```

Efter att en telnetuppkoppling initierats så kan klienten på HOST1 skicka kommandon till HOST2 som exekverar dessa på sin maskin och sedan vidarbefodrar resultatet tillbaka till HOST1 (Fig. 1).

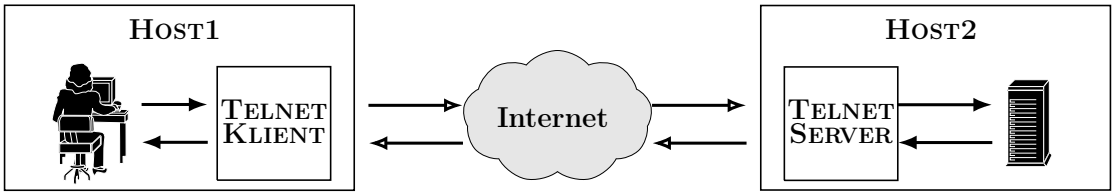

Figure 1: Terminalemulering över ett nätverk med Telnet.

I denna labb ska du implementera din egen Telnetklient och Telnetserver i programmeringsspråket C. Tillhörande uppgiften finns källkod som implementerar en "hello-world"-klient och "hello-world"-server i C för att hjälpa dig att komma igång med C-programmeringen. Din uppgift är att utöka både "hello-world"-klienten och "hello-world" servern så att de implementerar delar av Telnet protokollet. I slutet på uppgiften kommer du att kunna använda ditt program för att logga in på en kollegas server.

Innehållsförteckning

1 Förberedelser **3**

2 Mål **3**

3 Bakgrund **3**
3.1 Socketprogrammering i C 101 . . . 3
3.1.1 Steg 1: Skapa en TCP-Socket i C . . . 4
3.1.2 Steg 2: Öppna en TCP-Connection med din TCP-Socket . . . 4
3.1.3 Steg 3: Använd din TCP-Connection för att överföra data . . . 6
3.1.4 Steg 4: Specificera Kommunikationsprotokollet . . . 7
3.1.5 Något att tänka på: Minneshantering i C . . . 7

4 Uppgift **8**
4.1 Exempelexekvering . . . 10
4.1.1 Exempelutskrift från Klienten . . . 10
4.1.2 Exempelutskrift från Servern . . . 11
4.2 Debugging . . . 11
4.2.1 Exempelutskrift från Klienten med Debugging . . . 11
4.2.2 Exempelutskrift från Servern med Debugging . . . 12

5 Testkörning **14**

6 Bonusuppgift **14**

7 Kontrollfrågor **14**

A Exempelkod för att komma igång **15**
A.1 Kompilering . . . 15
A.2 Exekvering . . . 15
A.3 Exempelutskrift för Hello-World Klient-Server . . . 15

1 Förberedelser

- Läs Kapitel 4 och 5 om processhantering i UNIX i boken "Operating Systems: Three Easy Pieces" [1].
- Bekanta dig med Telnet genom att skumma igenom dess protokollspecifikation: `https://tools.ietf.org/html/rfc854`.
- Läs på om socketprogrammering i C online, exempelvis här: `https://www.thegeekstuff.com/2011/12/c-socket-programming/`.

2 Mål

Målet med uppgiften är att du ska:

- Lära dig om terminalemulering och hur Telnetprotokollet fungerar.
- Förvärva en förståelse om klient-server modellen.
- Öva på socketprogrammering i C.
- Reflektera över säkerhetsbrister i Telnet.

3 Bakgrund

Telnet använder som standard port 23 och använder TCP som dess underliggande transportprotokoll. Telnet har, historiskt sett, varit ett vanligt förekommande protokoll för terminalemulering i datornätverk. Dock så används Telnet nuförtiden mer sällan då det är ett protokoll med erkända säkerhetsbrister. Även om man fortfarande kan hitta Telnet på gamla servrar och switchar så har det mer moderna protokollet, *Secure SHell* (SSH), till stor del tagit Telnets plats. Telnet är ett öppet protokoll och du kan ta del av protokollspecifikationen här: `https://tools.ietf.org/html/rfc854`. I denna uppgift är det inte nödvändigt att implementera Telnetprotokollet precis enligt protokollspecifikationen. Det är tillräckligt att implementera en minimal variant av Telnet i C som möjliggör enkel terminalemulering över ett nätverk.

För att lyckas med denna uppgiften krävs dels en förståelse om nätverkskoncept såsom TCP, klient-server modellen, terminalemulering, samt Telnet; och dels programmeringskunskaper i C. För att implementera din klient och server så kommer du att behöva använda TCP sockets i C. Se till att läsa på om socket programmering online innan påbörjan av denna labb, du kan exempelvis läsa om socketprogrammering här: `https://www.thegeekstuff.com/2011/12/c-socket-programming/`. Tillhörande uppgiften finns även exempelkod för att komma igång (se appendix A).

3.1 Socketprogrammering i C 101

En nätverkssocket är i programmeringssammanhang en abstraktion för programmering av applikationer som använder nätverkskommunikation. Nätverkssocketen används som

en ändpunkt i ett nätverk från vilket en dator kan kommunicera med andra datorer över nätverket.

3.1.1 Steg 1: Skapa en TCP-Socket i C

För att skapa en TCP socket i C så kan du inkludera `#include <sys/socket.h>` och använda API anropet `socket()`, se manualen för mer information [1].

```
/**
 * Creates a TCP socket for IpV4
 *
 * @return socket file descriptor (int)
 */
int create_socket() {
    int sockfd;
    int opt = 1;
    //AF_INET means to use IPv4
    //SOCK_STREAM means to use TCP
    //0 is the protocol value for IP header (default)
    if ((sockfd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
        perror("socket creation failed");
        exit(1);
    }

    return sockfd;
}
```

3.1.2 Steg 2: Öppna en TCP-Connection med din TCP-Socket

Nästa steg, efter att du har skapat en TCP-socket, är att öppna en *TCP connection* till en mottgare. När du utvecklar ditt program så kan det vara praktiskt att öppna en TCP connection med ett annat program på samma dator, men du kan likaväl öppna en TCP connection med ett program på en dator på andra sidan världen, så länge du vet IP-adressen. Telnet använder klient-server modellen där en server väntar på inkommande connections från klienter. Nedan följer exempelkod för att programmera ett C program till att agera server och lyssna på inkommande TCP connections.

```
/**
 * Accepts incoming client connection on the given socket
 *
 * @param sockfd the server socket file descripto
 * @param sock_addr struct describing the IP address to
          listen for incoming connections on
 * @return a file descriptor for the socket of the accepted connection
 */
 int accept_incoming_connections(int sockfd,
```

[1] http://man7.org/linux/man-pages/man2/socket.2.html

```
                                struct sockaddr_in sock_addr) {
    printf("[Server] Waiting for incoming connections.. \n");

    int addrlen = sizeof(sock_addr);
    //Extracts the first connection request from
    //the queue of pending connections
    //for the listening socket.
    //Returns a file descriptor to the new socket. At this point,
    //the connection is established between client and
    //server and they are ready to transfer data.
    int client_socket;
    if ((client_socket = accept(sockfd, (struct sockaddr *) &sock_addr,
                                (socklen_t *) &addrlen)) < 0) {
        perror("[Server] failure");
        exit(-1);
    }
    printf("[Server] Accepted client connection\n");
    return client_socket;
}
```

Givet att serverprogrammet har startats så kan klienten koppla upp sig mot servern genom att använda systemanropet `connect()` och sin TCP-socket:

```
/**
 * Connects the socket to the server specified in server_address
 *
 * @param sockfd the socket to connect with
 * @param server_address the address of the host to connect with
 * @return 0 if successful
 */
int connect_to_server(int sockfd, struct sockaddr_in server_address) {
    printf("[Client] Connecting to server \n");
    //Connects is a system call that connects
    //the socket to the specified address and port
    int connect_res = connect(sockfd,
                              (struct sockaddr *) &server_address,
                              sizeof(server_address));
    if (connect_res < 0) {
        printf("[Client] Connection failed\n");
        return 1;
    }
    printf("[Client] Connected successfully with server \n");
    return 0;
}
```

3.1.3 Steg 3: Använd din TCP-Connection för att överföra data

När väl en TCP connection öppnats från din TCP-socket så kan socketen användas i samband med systemanropet `send()` för att överföra data mellan klienten och servern:

```
/**
 * Sends a command to the Telnet server over the socket
 *
 * @param sockfd the socket
 * @param msg  the mesage to send
 */
void socket_send_msg(int sockfd, char msg[]) {
    if (DEBUG) {
      printf("[Client] Sending command");
    }
    //Send system call, sends a message over the the given socket
    //0 in the last argument is a default flag.
    send(sockfd, msg, strlen(msg), 0);
    if (DEBUG) {
        printf("[Client] Command sent successfully to the server\n");
    }
}
```

Servern kan då ta emot meddelandet med hjälp av systemanropet `read()`:

```
/**
 * Reads a message of size command_size into the
 * given buffer from the given socket
 *
 * @param client_socket socket to read from
 * @param buffer the buffer to read the message into
 * @param command_size the size of the command to read
 * @return 0 if connection was closed, otherwise 1
 */
 int receive_socket_msg(int client_socket, char *buffer,
                        int command_size) {
    //System call for reading from a
    //file descriptor (a socket in this case)
    //On success, the number of bytes read is
    //returned (zero indicates end of file)
    int read_res = read(client_socket, buffer, command_size);
    if (read_res == 0) { //Connection closed
        printf("[Server] Connection lost \n");
        return read_res;
    } else {
        buffer[command_size] = '\0'; // Append NUL character
        if(DEBUG) {
            printf("[Server] Read %d bytes successfully \n", read_res);
```

```
        }
        printf("[Server] %s \n", buffer);
        return 1;
    }
}
```

3.1.4 Steg 4: Specificera Kommunikationsprotokollet

När den basala funktionaliteten är på plats och det är möjligt att skicka meddelanden över din socket så är nästa steg att specificera ett *protokoll* för kommunikationen. När och hur ska meddelanden skickas? När ska klienten skicka ett meddelande och när ska servern ta emot ett meddelande och vice-versa? Hur stora (antal bytes) ska meddelanden som skickas vara? Hur termineras en TCP uppkoppling på ett kontrollerat sätt?

Telnetprotokollet följer den generella klient-server modellen där klienten är den som förväntas initiera kontakten med servern. Efter att Telnetklienten och Telnetservern etablerat en uppkoppling så skickar Telnetklienten en serie med kommandon till Telnetservern som tolkar kommandona sekventiellt och returnerar resultaten till klienten (Fig. 2).

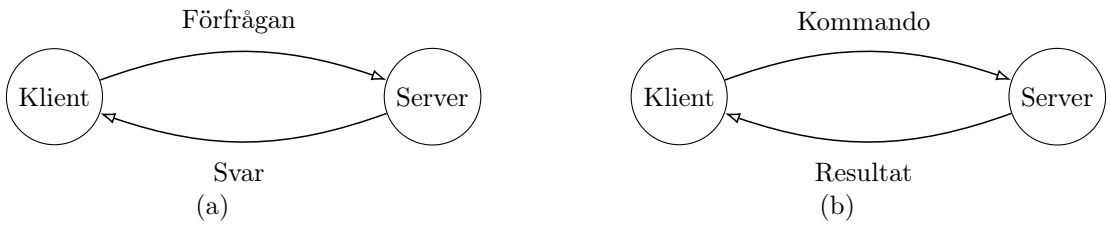

Figure 2: En generisk klient-server modell (Fig. 2a) och Telnet's klient-server modell (Fig. 2b)

3.1.5 Något att tänka på: Minneshantering i C

Eftersom att C har explicit minneshantering så uppstår ett problem när man programmerar med sockets som skickar och tar emot meddelanden av *dynamisk storlek*. Tänk dig följande scenario. Om din Telnetklient ibland skickar ett kommando av storlek 10 bytes och ibland ett kommando av storlek 1000 bytes, och ibland ett kommando av storlek 10000 bytes, hur många bytes måste Telnetservern allokera för att kunna ta emot dessa meddelanden? En naiv lösning är att allokera en stor buffer med en storlek som garanterat är större än det största kommandot som kan tänkas skickas av klienten (10000 i detta fallet). Det är dock ingen vidare hållbar lösning och den slösar minne. En bättre lösning, som är praxis för socketprogrammering i C, är att avsändaren först skickar längden på det nästföljande meddelandet och sedan skickar själva meddelandet. Det ger servern en chans att allokera minne av precis rätt storlek innan den läser det nästföljande meddelandet (Fig. 3).

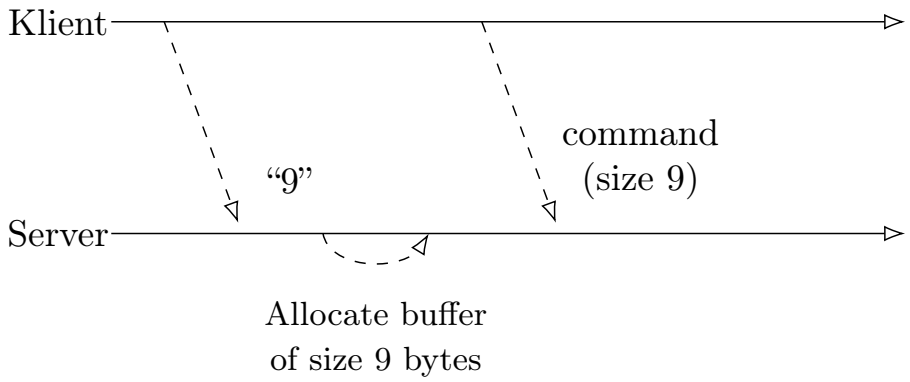

Figure 3: Kommunikationsprotokoll för dynamisk minneshantering i C.

Beakta följande exempel; Telnetklienten vill skicka ett meddelande $\mathcal{X}$ med storlek 19 bytes. Antag att klienten och servern har kommit överens tidigare om ett protokoll där klienten först skickar ett kort meddelande av storlek 4 bytes som inkluderar ett heltal $\in \mathbb{N}$ som definierar storleken på nästföljande meddelande. Då kan konversationen mellan klienten och servern gå till som följande:

1. Servern allokerar en buffer av storlek 4 bytes och väntar på ett meddelande från klienten.
2. Klienten skickar ett meddelande "19" till servern.
3. Servern tar emot meddelandet från klienten, tolkar meddelandet som ett heltal (integer av 4 bytes) "19" och allokerar en buffer av storlek 19 bytes och väntar på nästa meddelande.
4. Klienten skickar sitt meddelande av storlek 19 bytes.
5. Servern tar emot meddelandet av 19 bytes, tolkar meddelandet, och utför sedan den önskade operationen.

4 Uppgift

Du har friheten att själv välja hur du vill implementera din Telnetklient och Telnetserver. Det viktiga är att din implementation har den rätta funktionaliteten (se Sektion 5 "Testkörning"). Nedan följer *ett exempel* på hur Telnetservern och klienten kan tänkas fungera.

1. **Telnetservern startas och lyssnar på inkommande connections på TCP port 23 genom att exekvera server-binären:**
   - `./telnet_server`
   - Notera: det är möjligt att port 23 är upptagen på din maskin eller att du får ett `Permission denied` när du försöker binda en socket till port 23, välj då en annan port som är ledig, exempelvis port 9999.

2. **Klienten kopplar upp sig mot servern genom att exekvera klient-binären från kommandoraden:**
   - `./telnet_client <server_hostname>`
   - Tips: använd systemanropet `connect()`[2] i din C-kod för klienten.
3. **Efter att ha accepterat klientens connection så väntar servern på att ta emot längden av nästa kommando.**
   - Tips: använd systemanropet `accept()`[3] i din C-kod för servern.
4. **Klienten väntar på att användaren matar in nästa kommando (Tips: använd C-funktionen `fgets()`[4] (samma funktion som du använde i kommandotolk labben om du valde att implementera den i C).**
5. **Klienten tolkar användarens kommando och skickar längden på kommandot (antalet bytes) till servern.**
   - Tips: använd systemanropet `send()`[5] in din C-kod för att skicka meddelanden över sockets.
6. **Klienten skickar kommandot som matats in av användaren till servern.**
7. **Om kommandot var `exit` så stänger klienten sin socket till servern och terminerar programmet.**
8. **Servern tar emot och tolkar längden på kommandot.**
   - Tips: använd systemanropet `read()`[6] i din C-kod för att ta emot meddelanden över sockets.
9. **Om längden på kommandot är > 0 så allokerar servern en buffer med den angivna storleken för kommandot och tar emot kommandot.**
   - OBS: C använder sig av NULL-terminerade strängar. Strängar i C avslutas med karaktären NUL (`'\0'`). Om du inte räknar med NUL karaktären i storleken av kommandot så kommer din socket inte att skicka med NUL karaktären och du kan då behöva tillägga (append) NUL till kommandot på serversidan.
10. **Efter att servern tagit emot ett kommando med längd >0 av klienten så tolkar serven kommandot:**
    - Om kommandot är `exit` så stänger servern sin socket och terminerar programmet.
    - Om kommandot inte är `exit` så exekverar servern kommandot genom att använda C-funktionen `popen()`[7]. Efter att servern exekverat ett kommandot

[2] http://man7.org/linux/man-pages/man2/connect.2.html
[3] http://man7.org/linux/man-pages/man2/accept.2.html
[4] https://linux.die.net/man/3/fgets
[5] http://man7.org/linux/man-pages/man2/send.2.html
[6] http://man7.org/linux/man-pages/man2/read.2.html
[7] http://man7.org/linux/man-pages/man3/popen.3.html

med `popen()` så genomför den dessa tre steg i ordning:

(a) Servern skickar längden på resultatet av det exekverade kommandot till klienten.

(b) Servern skickar resultatet av det exekverade kommandot till klienten.

(c) Slutligen så skickar servern meddelandet "0" till klienten för att indikera att den har skickat hela resultatet till klienten. Det vill säga, ett meddelande med "0" fungerar som en "end-of-message" indikator.

11. **Klienten väntar på meddelanden från sin TCP socket efter att ha skickat iväg kommandot till servern. Om klienten tar emot en buffer på sin TCP socket med längd > 0 så tar klienten emot meddelandet och tolkar det som "längden" på nästa meddelande som ska tas emot.**

12. **Klienten allokerar en buffer av samma längd som den tog emot av servern och väntar på att meddelandet ska skickas från servern.**

13. **Repetera hela processen tills dessa att användaren matar in ett `exit` kommando.**

4.1 Exempelexekvering

Exempelutskrift som illustrerar hur ditt program ska fungera följer nedan.

4.1.1 Exempelutskrift från Klienten

```
kim@kim-vox:/home/kim/telnet_client$ ./telnet_client
Telnet Client Starting
[Client] Configuring server to connect to. Address: 127.0.0.1, port: 9999
[Client] Connecting to server
[Client] Connected successfully with server
$ ls
[Client] CMakeLists.txt
[Client] Makefile
[Client] Makefile~
[Client] cmake-build-debug
[Client] telnet_server
[Client] telnet_server.c
$ ls ../
[Client] 1_signals
[Client] 1_signals_2
[Client] 2_pipes
[Client] telnet_1
[Client] telnet_client
[Client] telnet_helpcode
[Client] telnet_server
$ ls ../../
```

```
[Client] README.md
[Client] ac_script
[Client] dekompilering
[Client] kommandotolk
[Client] signals_pipes_telnet
$ pwd
[Client] /Users/kimham/workspace/cybersoldat_labbar/signals_pipes_telnet/telnet_serve
$ exit
[Client] Connection lost
```

4.1.2 Exempelutskrift från Servern

```
kim@kim-vox:/home/kim/telnet_server$ ./telnet_server
Telnet Server Starting
[Server] Listening to port 9999
[Server] Waiting for incoming connections..
[Server] Accepted client connection
[Server] ls
[Server] ls ../
[Server] ls ../../
[Server] pwd
[Server] exit
[Server] exiting
[Server] Waiting for incoming connections.
```

4.2 Debugging

För att demonstrera steg för steg hur ditt program kan tänkas fungera så har vi testkört en implementation med debugutskrifter som förklarar exekveringen steg för steg, för vilket utskriften finns nedan.

4.2.1 Exempelutskrift från Klienten med Debugging

```
kim@kim-vox:/home/kim/telnet_client$ ./telnet_client
Telnet Client Starting
[Client] Configuring server to connect to. Address: 127.0.0.1, port: 9999
[Client] Connecting to server
[Client] Connected successfully with server
$ ls
[Client] Notifying server of the size of next command which is 2 bytes
[Client] Size of next message sent successfully to the server
[Client] Sending command: ls of length: 2 bytes
[Client] Command sent successfully to the server
[Client] Waiting for server to notify me about the size of the next message
[Client] Read 4 bytes successfully
[Client] Size of the next message from the server will be 15 bytes
[Client] Waiting for server to send a result message of size 15
```

```
[Client] Read 15 bytes successfully
[Client] CMakeLists.txt
[Client] Waiting for server to notify me about the size of the next message
[Client] Read 4 bytes successfully
[Client] Size of the next message from the server will be 9 bytes
[Client] Waiting for server to send a result message of size 9
[Client] Read 9 bytes successfully
[Client] Makefile
[Client] Waiting for server to notify me about the size of the next message
[Client] Read 4 bytes successfully
[Client] Size of the next message from the server will be 10 bytes
[Client] Waiting for server to send a result message of size 10
[Client] Read 10 bytes successfully
[Client] Makefile~
[Client] Waiting for server to notify me about the size of the next message
[Client] Read 4 bytes successfully
[Client] Size of the next message from the server will be 18 bytes
[Client] Waiting for server to send a result message of size 18
[Client] Read 18 bytes successfully
[Client] cmake-build-debug
[Client] Waiting for server to notify me about the size of the next message
[Client] Read 4 bytes successfully
[Client] Size of the next message from the server will be 14 bytes
[Client] Waiting for server to send a result message of size 14
[Client] Read 14 bytes successfully
[Client] telnet_server
[Client] Waiting for server to notify me about the size of the next message
[Client] Read 4 bytes successfully
[Client] Size of the next message from the server will be 16 bytes
[Client] Waiting for server to send a result message of size 16
[Client] Read 16 bytes successfully
[Client] telnet_server.c
[Client] Waiting for server to notify me about the size of the next message
[Client] Read 4 bytes successfully
[Client] Size of the next message from the server will be 0 bytes
$
```

4.2.2 Exempelutskrift från Servern med Debugging

```
kim@kim-vox:/home/kim/telnet_server$ ./telnet_server
Telnet Server Starting
[Server] Listening to port 9999
[Server] Waiting for incoming connections..
[Server] Accepted client connection
[Server] Waiting for client to notify me about the size of the next command
[Server] Read 4 bytes successfully
```

```
[Server] Size of the next message from the Client will be 2 bytes
[Server] Waiting for client to send a command of size 2
[Server] Read 2 bytes successfully
[Server] ls
[Server] sending result to client: CMakeLists.txt
[Server] Notifying client of the size of the next result-message, which is 15 bytes
[Server] Size of next message sent successfully to the client
[Server] Sending command result: CMakeLists.txt
 of length: 15 bytes to client
[Server] Command result sent successfully to the client
[Server] sending result to client: Makefile
[Server] Notifying client of the size of the next result-message, which is 9 bytes
[Server] Size of next message sent successfully to the client
[Server] Sending command result: Makefile
 of length: 9 bytes to client
[Server] Command result sent successfully to the client
[Server] sending result to client: Makefile~
[Server] Notifying client of the size of the next result-message, which is 10 bytes
[Server] Size of next message sent successfully to the client
[Server] Sending command result: Makefile~
 of length: 10 bytes to client
[Server] Command result sent successfully to the client
[Server] sending result to client: cmake-build-debug
[Server] Notifying client of the size of the next result-message, which is 18 bytes
[Server] Size of next message sent successfully to the client
[Server] Sending command result: cmake-build-debug
 of length: 18 bytes to client
[Server] Command result sent successfully to the client
[Server] sending result to client: telnet_server
[Server] Notifying client of the size of the next result-message, which is 14 bytes
[Server] Size of next message sent successfully to the client
[Server] Sending command result: telnet_server
 of length: 14 bytes to client
[Server] Command result sent successfully to the client
[Server] sending result to client: telnet_server.c
[Server] Notifying client of the size of the next result-message, which is 16 bytes
[Server] Size of next message sent successfully to the client
[Server] Sending command result: telnet_server.c
 of length: 16 bytes to client
[Server] Command result sent successfully to the client
[Server] Notifying client of the size of the next result-message, which is 0 bytes
[Server] Size of next message sent successfully to the client
[Server] Waiting for client to notify me about the size of the next command
```

5 Testkörning

När du har fått ditt eget program att fungera, para ihop dig med en kollega och låt en av er agera server och den andra agera klient. Till exempel, om rekryt 1 har IP X och rekryt 2 har IP Y och rekryt 2 agerar server så ska rekryt 2 exekvera:

```
./telnet_server
```

Rekryt 1 som ska då koppla upp sin klient mot rekryt 2s server:

```
./telnet_client IP Y
```

Om implementationerna är korrekta så ska klienten kunna exekvera i princip alla bash-kommandon på servern och få förväntat resultat. Det är tillräckligt att demonstrera för en handledare att kommandona `ls` och `pwd` ger förväntat resultat för att få godkänt på uppgiften.

6 Bonusuppgift

Gör Telnetservern flertrådad så att den kan hantera flera klienter parallellt. Tips: använd systemanropet `fork()` för att skapa nya child-processer.

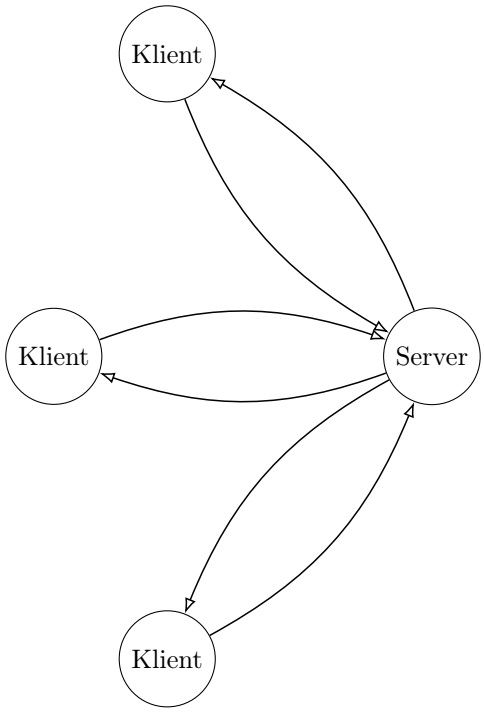

Figure 4: En flertrådad Telnetserver kan hantera flera klienter parallelt.

7 Kontrollfrågor

- Varför använder Telnet TCP istället för UDP? Är det möjligt att implementera ett protokoll med samma funktionalitet som Telnet fast över UDP?

- Varför tror du Telnet-protokollet utvecklades utan kryptering? Vad är fördelar och nackdelar med att använda krypterad kommunikation?
- Hur fungerar `popen()`? Hint: använd manualen (`man popen`)
- Varför måste servern skicka ett "end-of-message" meddelande till klienten efter att den skickat resultatet av ett kommando tillbaka till klienten? Hint: läs på om pipes och `popen()`.
- Vad innebär det att `read()` systemanropet är ett "blockerande" anrop?
- Hur fungerar Klient-server modellen inom datorkommunikation? Kan du nämna några andra kommunikationsmodeller? Hur skiljer de sig åt?

A Exempelkod för att komma igång

För att underlätta uppgiften och ge en snabbstart i socketprogrammering i C så har vi utvecklat en enkel klient-server i C som är kapabel att skicka och ta emot ett "Hello World" meddelande. Koden finns tillgänglig att laddas ner här: `https://gits-15.sys.kth.se/kimham/cybersoldat_labbar`. Det kan vara en bra idé att använda exempelprogrammen som grund för din egen implementation men det är inget krav. För att kompilera och exekvera exempelprogrammen, se instruktioner nedan.

A.1 Kompilering

För att kompilera och bygga både klient och server, använd kommandot:

```
make all
```

För att kompilera och bygga endast klienten använd kommandot:

```
make client
```

För att kompilera och bygga endast servern använd kommandot:

```
make server
```

A.2 Exekvering

För att starta servern, använd följande kommando efter att du har kompilerat:

```
./server
```

För att starta klienten, använd följande kommando efter att du har kompilerat:

```
./client
```

A.3 Exempelutskrift för Hello-World Klient-Server

Server

```
kim@kim-vox:/home/kim/telnet_helpcode$ make clean
rm -rf server client
kim@kim-vox:/home/kim/telnet_helpcode$ make all
gcc -o server server.c
gcc -o client client.c
kim@kim-vox:/home/kim/telnet_helpcode$ ./server
Telnet Server Starting
[Server] Listening to port 9999
[Server] Waiting for incoming connections..
[Server] Accepted client connection
[Server] Reading from socket..
[Server] Read 27 bytes successfully
[Server] Read the message: Hello from Enköping/Client
[Server] Sending msg: Hello from Stockholm/Server , of length: 27
[Server] Message sent successfully
[Server] Closing the client connection and waiting for the next connection
[Server] Waiting for incoming connections..
```

Klient

```
kim@kim-vox:/home/kim/telnet_helpcode$ ./client
Telnet Client Starting
[Client] Configuring server to connect to, address: 127.0.0.1, port: 9999
[Client] Connecting to server
[Client] Connected successfully with server
[Client] Sending msg: Hello from Enköping/Client , of length: 27
[Client] Message sent successfully
[Client] Reading from socket..
[Client] Read 27 bytes successfully
[Client] Read the message: Hello from Stockholm/Server
[Client] Closing the socket and exiting
```

Referenser

[1] Remzi H. Arpaci-Dusseau and Andrea C. Arpaci-Dusseau. *Operating Systems: Three Easy Pieces*. Arpaci-Dusseau Books, 1.00 edition, August 2018.